모바일프로그래밍 프로젝트 제안서

1. 프로젝트 개요

달무리버스는 지도에서 정류장을 골라 호출하면, 여러 대의 자율주행버스가 승객을 합승시키며 전체 승객이 가장 빨리 도착하도록 실시간으로 배차하는 안드로이드 앱입니다.

항목	내용
프로젝트명	심야 합승형 자율주행버스 실시간 호출·배차 앱 (달무리버스)
개발 유형	팀
팀원	윤재열 (20223086), 장정훈 (20223093)
개발 환경	Android Studio, Kotlin, Jetpack Compose
시범지역 / 운영 시간	서울 관악구 정류장 20~30 개 / 심야 00 시~05 시

전제 — 자율주행버스가 상용화되어 시범지역에 여러 대 운행 중이라고 가정합니다.
본 프로젝트는 차량이 아닌 호출·배차 서비스(앱)를 개발하며,
차량 운행은 앱 안의 관제 모드에서 시뮬레이션으로 구현합니다.

***)'달무리'는 달 주위에 생기는 빛의 고리로, 어두운 귀갓길을 빛으로 감싸는 서비스라는 뜻을 담았습니다.**

2. 목적 및 필요성

고정 노선 심야버스로는 해결되지 않는 '돌아가는 귀갓길'과 '집까지 남은 긴 도보'를,
부르면 오는 합승형 자율주행버스로 줄이는 것이 목적입니다.

해결하고자 하는 문제

- **긴 배차 간격과 적은 노선** — 심야에는 배차 간격이 길고 노선 수가 적어, 목적지까지 빙 돌아가거나 오래 기다려야 합니다.
- **집에서 먼 하차 정류장** — 심야버스가 집 근처를 지나지 않으면 먼 정류장에 내려 조명·유동인구가 적은 이면도로를 오래 걸어야 하고, 이는 귀갓길 불안으로 이어집니다.
- **대체 수단의 한계** — 택시는 1 인 이용 시 비용 부담이 크고, 인력 기반 귀가 지원(안심귀가 스카우트)은 확장에 한계가 있습니다.

해결 방향

설계 원칙	방법	기대 효과
수요응답형 (DRT)	고정 노선 대신 호출이 들어온 정류장만 들르는 자율주행버스를 여러 대 운행	돌아가는 경로와 긴 배차 간격 해소
정류장 기반 승하차	출발·도착을 기존 버스 정류장으로 한정	조명·쉼터 등 기존 인프라 활용, 집 주소를 남기지 않아 거주지 노출 방지, 이면도로 주행이 없어 자율주행 규제 부담 감소
합승 최적 배차	한 버스에 여러 승객을 태우되, 방향이 다른 승객은 다른 버스가 태우도록 배정	전체 승객의 도착시간 최소화, 차량 효율 향상

배경 및 사례 분석

심야 자율주행버스는 서울에서 이미 고정 노선으로 달리고 있고, 호출형 합승버스(DRT)는 경기도에서 검증됐지만 대기시간이 핵심 과제로 남아 있습니다.
본 프로젝트는 두 흐름을 결합합니다.

사례	내용	시사점
서울 심야 자율주행버스 (심야 A21)	2023 년 12 월 4 일 운행 시작. 평일 23:30~05:10, 합정~동대문 중앙버스전용차로 9.8km 를 70 분 간격으로 순환, 특별안전요원 등 2 명 동승	심야 자율주행은 이미 현실화 단계. 다만 고정 노선이라 노선 밖은 여전히 사각지대
서울시 자율주행버스 확대 계획	2026 년 8 월 발표. 2028 년 실증, 2029 년 대량 도입, 2030 년까지 500 대. 새벽·심야 시간대와 중앙버스전용차로부터 적용. 10 월에는 레벨 4 자율주행 시내버스 업무협약 체결	차량 공급이 늘수록 '어떻게 효율적으로 배차할지'가 서비스 품질을 좌우
경기도 똑버스(DRT)	정해진 노선 없이 운행되는 수요응답형 버스. 2023 년 파주·운정·교하에 11 인승 차량 15 대 운행	본 앱이 구현할 합승 배차의 실제 운영 사례
똑버스 대기시간 문제	2023 년 운정·교하 대기 16.5 분, 불만 대다수가 긴 대기와 배차 실패. 2026 년 7 월에도 도의회가 평균 약 15 분인 대기시간의 감축 목표를 요구	DRT 의 성패는 대기시간. 최대 대기시간 제약과 효율적인 배차 알고리즘이 필요

시사점 — 본 앱은 ① 출발·도착을 기존 정류장으로 한정해 문 앞 호출보다 배차를 단순하게 만들고, ② '대기 15 분 이내 · 기존 승객 지연 5 분 이내' 제약을 둔 배차 알고리즘으로 똑버스가 겪은 대기시간 문제를 해결합니다.

3. 주요 기능

핵심은 두 가지입니다. 여러 버스와 여러 승객을 동시에 다루는 실시간 합승 배차, 그리고 호출부터 하차까지 지도 위에서 끊김 없이 보여 주는 실시간 추적입니다.

핵심 기능 1 — 실시간 합승 배차

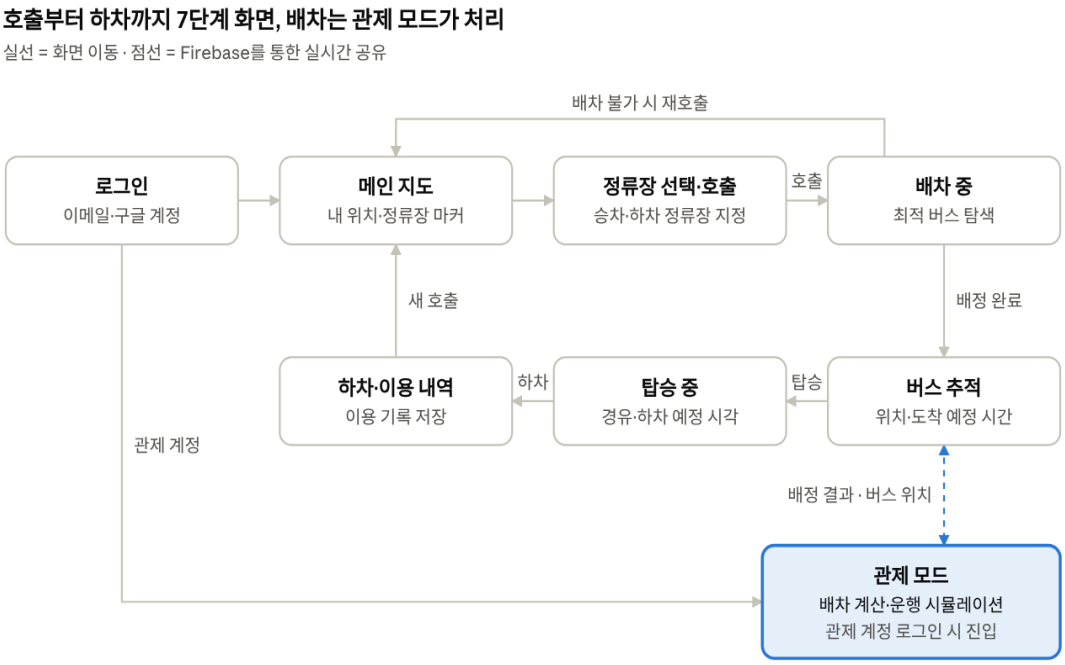
- 여러 대의 버스가 동시에 운행하며, 새 호출이 들어오면 운행 중인 모든 버스의 남은 경로에 이 승객의 승차·하차 정류장을 끼워 넣어 봅니다.
- 전체 승객의 하차 시각 합이 가장 적게 늘어나는 버스와 위치에 배정합니다.
- 승차 위치가 비슷해도 목적지가 반대라 기존 승객이 크게 돌아가야 하면, 다른 버스가 태우도록 배정됩니다. 알고리즘 구현은 7 장에서 설명합니다.

제약조건	기준	위반 시
신규 승객 대기시간	호출 후 15 분 이내 승차	해당 배정안 제외
기존 승객 도착 지연	안내한 하차 시각 대비 5 분 이내	해당 배정안 제외
차량 정원	모든 구간에서 탑승 인원 ≤ 정원	해당 배정안 제외

핵심 기능 2 — 지도 기반 정류장 호출 및 실시간 버스 추적

- 지도에 시범지역 정류장을 마커로 표시하고, 내 위치에서 가까운 정류장을 승차지로 추천합니다. 하차지는 지도나 즐겨찾기에서 고릅니다.
- 배정되면 버스의 실시간 위치, 경로선, 승차 정류장 도착까지 남은 시간, 하차 예정 시각을 보여 줍니다.
- 호출 상태(배차 중 → 배정 완료 → 접근 중 → 탑승 중 → 하차 완료)에 따라 하단 시트가 바뀝니다. 흐름은 4 장 다이어그램에 정리했습니다.
- 합승으로 경로가 바뀌면 예상 시각이 자동으로 갱신되고, 추가된 경유 정류장과 바뀐 시각을 알려 줍니다.

4. 앱 흐름도



앱 화면 흐름 · 사용자 화면 7 개 + 관제 모드

사용자는 한 지도 화면 위에서 7 단계를 거치고, 배차 불가면 메인 지도로 돌아가 다시 호출합니다.

관제 모드는 같은 앱을 관제 계정으로 열어 배차와 운행 시뮬레이션을 맡고, 그 결과는 Firebase 를 통해 사용자 화면에 실시간으로 공유됩니다.

5. UI 프로토타입

지도 한 장 위에서 하단 시트가 호출 단계별로 바뀌는 구조이며,

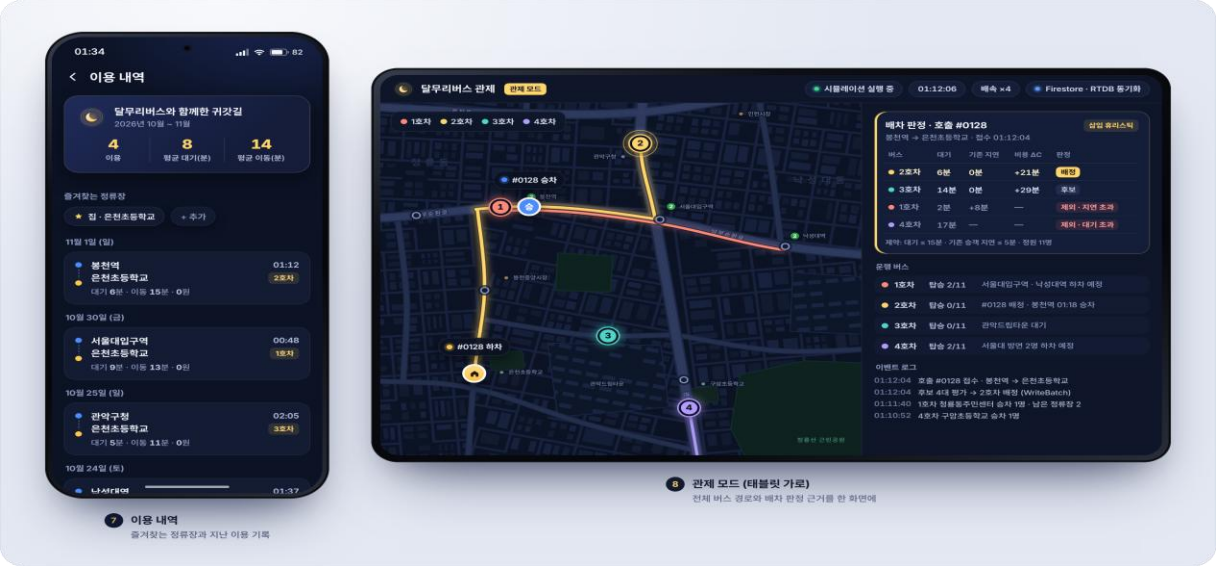
심야에 쓰는 앱이라 다크 테마를 기본으로 설계했습니다.



① 로그인 · ② 메인 지도 · ③ 정류장 선택·호출 — 내 위치에서 가까운 조명 정류장을 추천하고, 승차·하차와 요금을 확인한 뒤 호출합니다.



④ 배차 완료·버스 추적 · ⑤ 탑승 중 · ⑥ 하차 완료 — 배정된 2 호차가 다가오고, 합승으로 하차 시각이 바뀌면 바로 알려 줍니다.



⑦ 이용 내역 · ⑧ 관제 모드 — 버스 4 대의 경로와 호출의 배차 판정을 한 화면에 보여 줍니다.

디자인 원칙

원칙	적용
다크 테마 기본	야간 눈부심을 줄이고, 네이버 지도 야간 모드와 색을 맞춤
색으로 역할 구분	승차 = 블루, 하차= 달빛 골드, 내 버스 = 흰색 + 달무리 링
단일 지도 + 하단 시트	하단 시트만 단계별로 교체해 화면 전환을 최소화
핵심 숫자 우선	'도착까지 N 분'과 '하차 예정 시각'을 가장 크게 표시
안심 요소 노출	정류장 대기 안내, 귀가 공유 버튼을 탑승 흐름 안에 배치

구현·검증 계획

- **Jetpack Compose Preview** — 정류장 선택 카드, 호출 버튼, 배차 정보 하단 시트, 버스 도착 안내 카드, 이용 내역 항목을 컴포넌트별로 렌더링하고, 호출 상태(UiState)마다 @Preview 를 두어 상태별 화면을 바로 확인합니다.
- **Figma** — 위 시안을 Figma 컴포넌트로 옮겨 세부 시안을 다듬은 뒤 링크를 첨부할 예정입니다.
- **손그림 스캔** — 위 시안으로 대체합니다.

6. 사용 기술

구분	기술	용도
Android Jetpack	Jetpack Compose	선언형 UI, 다크 테마, 단계별 하단 시트
Android Jetpack	Navigation-Compose	화면 전환(로그인 · 지도 · 이용 내역 · 관제 모드)
Android Jetpack	ViewModel, StateFlow	화면 상태 관리, 배차 상태·버스 위치 실시간 관찰
Android Jetpack	Hilt	Repository·데이터 소스 교체와 테스트를 쉽게
데이터 저장소	Room	정류장 목록, 정류장 간 소요시간 표·경로선
데이터 저장소	Preferences DataStore	사용 모드(사용자/관제), 온보딩 여부 등 간단한 설정
네트워크	Retrofit2, OkHttp	카카오모빌리티 길찾기 REST API 통신
Firebase	Authentication	이메일·구글 계정 로그인, 관제 계정 구분
Firebase	Cloud Firestore	호출 요청, 배차 결과, 버스별 남은 경로, 이용 기록
Firebase	Realtime Database	버스 실시간 위치(2 초 주기 덮어쓰기)
지도	네이버 지도 Android SDK	정류장·버스 마커, 경로선, 야간 모드 지도
경로·소요시간	카카오모빌리티 API	정류장 쌍별 도로 경로와 소요시간을 수집해 저장
공공데이터	서울시 버스정류장 위치정보	시범지역 정류장명·좌표(CSV → Room 초기 데이터)
위치	Fused Location Provider	내 위치 기준 가까운 정류장 추천

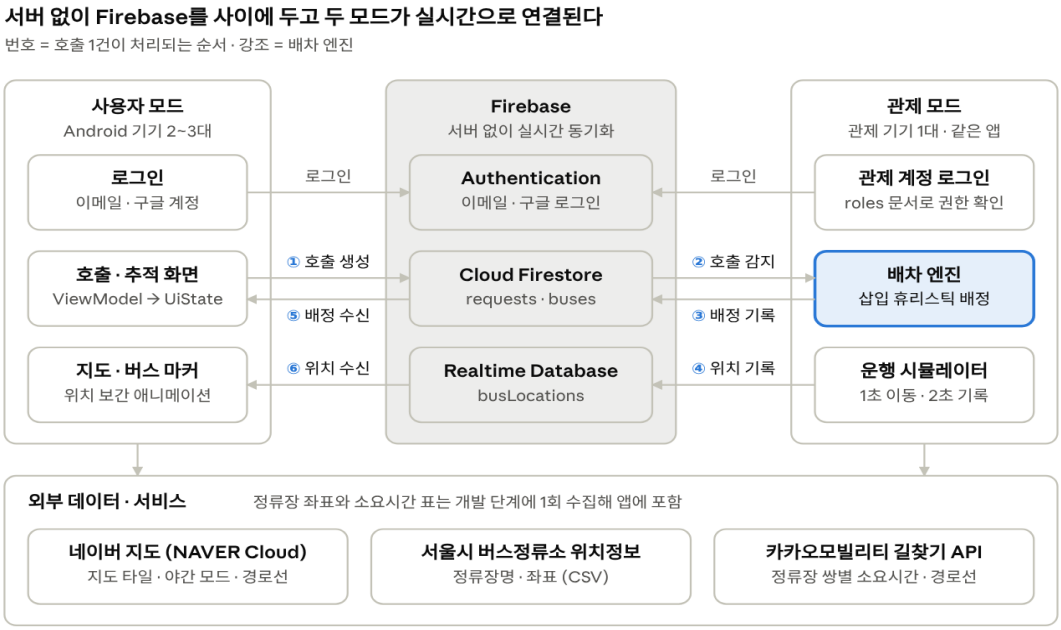
기술 선택 이유

- **네이버 지도** — 국내 도로 데이터가 정확하고 Compose 래퍼를 쓸 수 있습니다.
- **Firestore + Realtime Database 분리** — Firestore 무료 한도 안에서 개발과 시연을 모두 진행합니다.
- **Hilt + Repository** — 배차 엔진과 데이터 소스를 인터페이스로 분리해, 사용자 모드와 관제 모드가 같은 코드를 공유하고 가짜 저장소로 테스트할 수 있습니다.

7. 시스템 아키텍처 및 구현 방법

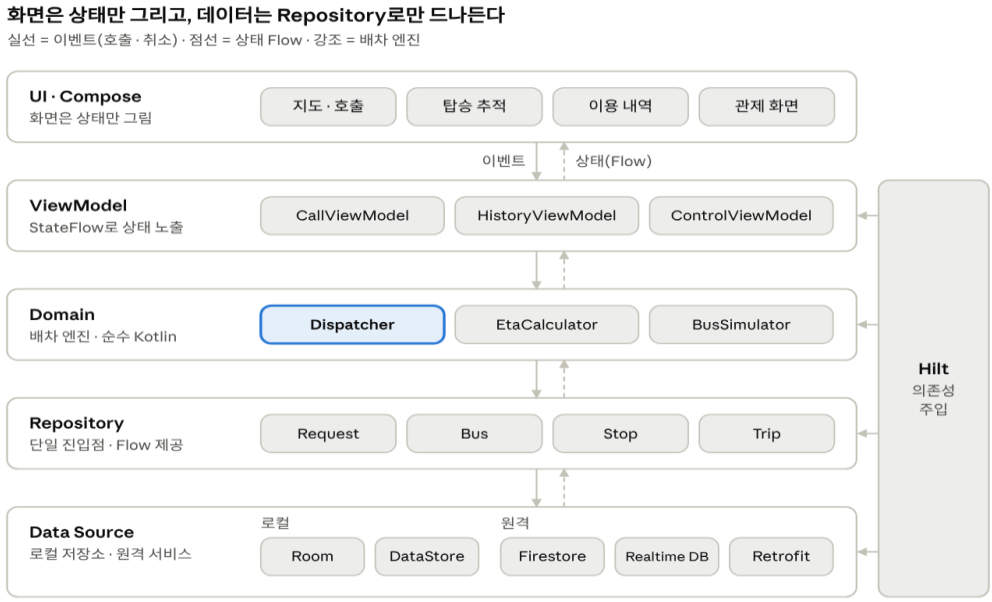
별도 서버 없이, 사용자 모드와 관제 모드(같은 앱의 별도 모드)가 Firebase 를 사이에 두고 호출·배차 결과·버스 위치를 주고받는 서버리스 구조입니다.

7.1 시스템 구성도



1. **호출 생성** — 사용자 앱이 Firestore `requests` 에 호출 문서를 만듭니다(status = REQUESTED).
2. **호출 감지** — 관제 모드가 실시간으로 새 호출을 받아 생성 순서대로 하나씩 처리합니다.
3. **배정 기록** — 배차 엔진의 결과(배정 버스·예상 시각)와 그 버스의 남은 경로를 WriteBatch 로 함께 기록합니다.
4. **위치 기록** — 버스 위치를 2 초마다 Realtime Database 에 덮어씁니다.
5. **배정 수신** — 앱이 호출 문서의 변화를 받아 하단 시트를 '배정 완료'로 바꿉니다.
6. **위치 수신** — 배정된 버스의 위치만 구독하고, 2 초 간격 위치 사이를 보간 애니메이션으로 이어 지도에 표시합니다.

7.2 앱 내부 구조 — MVVM + Repository



앱 내부 계층 · UI → ViewModel → Domain → Repository → Data Source, Hilt 주입

```
com.dalmuri.bus
├─ ui/                  Compose 화면 + ViewModel (기능별 패키지)
│  ├─ auth/  map/  ride/  history/  control/
│  └─ theme/  component/
├─ domain/              순수 Kotlin — Android 의존성 없음
│  ├─ model/            Stop · Request · Bus · Assignment
│  ├─ dispatch/         Dispatcher(삽입 휴리스틱) · EtaCalculator
│  └─ sim/              BusSimulator(관제 모드 운행)
├─ data/
│  ├─ repository/       Request · Bus · Stop · Trip Repository
│  ├─ local/            Room(Dao · Entity) · DataStore
│  └─ remote/           Firestore · RealtimeDB · KakaoMobilityApi(Retrofit)
└─ di/                  Hilt 모듈
```

- **단방향 데이터 흐름** — 화면은 ViewModel 에 이벤트(호출·취소)만 보내고, ViewModel 이 내보내는 UiState(StateFlow)만 그립니다.
- **리스너의 Flow 변환** — Firestore·Realtime Database 리스너는 Repository 에서 callbackFlow 로 감싸 Flow 로 내보냅니다.
- **테스트하기 쉬운 배차 엔진** — 배차 엔진은 Android 의존성이 없는 순수 Kotlin(domain)이라 JUnit 으로 바로 검증합니다.

7.3 데이터 모델

저장소 · 경로	주요 필드	쓰는 쪽 → 읽는 쪽
Firestore `requests/{requestId}`	uid, pickupStopId, dropoffStopId, status, busId, pickupEta, dropoffEta, 단계별 시각	사용자 앱(생성·취소) ↔ 관제 모드(배정·상태 갱신)
Firestore `buses/{busId}`	name, capacity, onboard, remainingStops[stopId · 승차/하차 · requestId · eta]	관제 모드 → 사용자 앱
Realtime DB `/busLocations/{busId}`	lat, lng, bearing, updatedAt	관제 모드(2 초 주기) → 사용자 앱
Room(앱 로컬)	stops, travel_times, favorites, recent_destinations, trip_history	앱 내부 전용

- **호출 상태** — `REQUESTED → ASSIGNED → APPROACHING → ONBOARD → COMPLETED`, 예외는 `REJECTED`(배차 불가)와 `CANCELED`(취소)입니다. 이용 내역은 완료된 내 호출을 조회해 Room 에 캐시합니다.
- **보안 규칙** — 사용자는 본인 호출만 생성·조회·취소하고, 배정과 상태 변경은 관제 계정(roles 문서로 지정)만 할 수 있습니다.

7.4 배차 알고리즘 — 삽입 휴리스틱(Cheapest Insertion)

매번 전체 최적해를 푸는 대신, 새 호출을 기존 경로 중 손해가 가장 적은 자리에 끼워 넣습니다.

각 버스는 앞으로 들를 정류장 순서(남은 경로)를 가집니다. 예: 버스 A = [S3 하차 → S5 하차]

1. 새 호출(승차 P, 하차 D)이 오면 모든 버스에 대해 P 와 D 를 끼워 넣을 수 있는 모든 위치를 시도합니다. P 는 항상 D 보다 앞에 둡니다.
2. 각 경우의 비용은 전체 승객 하차 시각 합의 증가량, 즉 신규 승객의 도착 시각과 기존 승객들의 지연 합입니다.
3. 비용이 가장 작은 버스·위치에 배정하고 그 버스의 경로를 갱신합니다. 남는 경우가 없으면 배차 불가로 안내합니다.

비용 ΔC = (삽입 후 승객의 하차 예정 시각 합) − (삽입 전 기존 승객의 하차 예정 시각 합)

```
/** 새 호출을 비용 증가가 가장 작은 버스.위치에 끼워 넣는다 */
fun assign(req: Request, buses: List<Bus>, t: TravelTimeTable): Assignment? {
    var best: Assignment? = null
    for (bus in buses) {
        val route = bus.remainingStops        // 남은 경로
        val before = sumArrival(bus, route, t)
        for (i in 0..route.size) {            // 승차 위치
            for (j in i..route.size) {        // 하차 위치(승차 뒤)
                val newRoute = route
                    .insert(i, req.pickup)
                    .insert(j + 1, req.dropoff)
                // 대기 15 분 · 기존 승객 지연 5 분 · 정원 확인
                if (!isFeasible(bus, newRoute, req, t)) continue
                val cost = sumArrival(bus, newRoute, t) - before
                if (best == null || cost < best.cost) {
                    best = Assignment(bus, newRoute, cost)
                }
            }
        }
    }
    return best   // null 이면 배차 불가
}
```

예시 — 같은 곳에서 타도 방향이 다르면 다른 버스. 버스 A 는 동쪽으로 승객을 태우고 가는 중이고, 버스 B 는 빈 차로 근처에 있습니다. A 근처 정류장에서 서쪽 목적지로 가는 호출이 들어온 경우입니다.

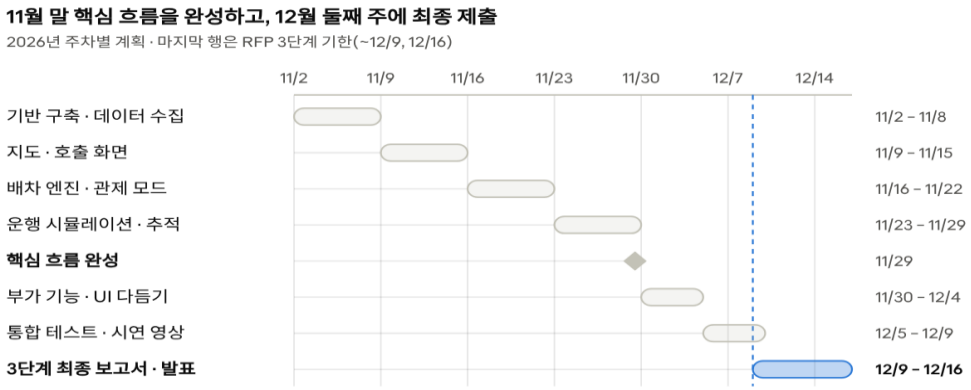
8. 역할 분담 및 개발 일정

핵심 흐름(호출 → 배정 → 버스 접근 → 하차)을 11 월 넷째 주까지 먼저 완성해 중간 점검에서 보여 주고, 이후 부가 기능과 시연을 마무리합니다.

역할 분담

역할	담당 내용
프론트엔드	Jetpack Compose 화면 구현(로그인, 메인 지도, 호출, 배차·추적, 탑승, 이용 내역, 관제 모드), 네이버 지도 연동과 정류장·버스 마커·경로선 렌더링, 호출 상태별 하단 시트와 인터랙션
백엔드/DB	Firestore·Realtime Database 데이터 모델과 보안 규칙, Firebase 인증, 합승 배차 엔진과 단위 테스트, 관제 모드·운행 시뮬레이터, 정류장 데이터·소요시간 표 구축
문서화 및 발표	배경·사례 정리, 제안서·중간 점검·최종 보고서, 발표 자료 제작

개발 일정



시연 계획

- 기기 2~3 대에서 동시에 호출해, 같은 방향 승객의 합승 배정과 반대 방향 승객의 다른 버스 배정을 지도로 보여 줍니다.
- 관제 모드 화면을 함께 띄워 버스별 남은 경로와 배차 판정 근거를 실시간으로 비교합니다.

참고 자료

- 서울시 미디어허브 — 세계 최초 '심야 자율주행버스' 운행시간·노선
- 서울신문 — 2030 년 서울에 자율주행버스 500 대 투입(2026.8.21)
- 머니투데이 — '레벨 4' 자율주행 시내버스 500 대 도입(2026.10.1)
- 서울신문 — 파주 운정·교하 똑버스 5 대 증차(2023.10.20)
- 서울신문 — 경기도의회, 똑버스 운영 재점검 촉구(2026.7.21)
- 더팩트 — '1 인 가구' 가장 많은 관악구(2025.4.13)
- Kakao Developers — 쿼터
- 카카오 데브톡 — 자동차 길찾기 API 사용 문의 답변
- NAVER Cloud — Maps 상품·요금
- 네이버 지도 Android SDK — 지도 유형·야간 모드
- naver-map-compose (GitHub)
- Cloud Firestore — Quotas and limits
- 선행 분석: 서울시 노선·정류장 데이터 기반 N 버스 사각지대 분석